

The proofgraph package

Pierre Senellart
pierre@senellart.com

2026/05/31 v0.1.0

Abstract

The **proofgraph** package automatically produces a graph of the dependencies between the results (theorems, lemmas, propositions) of a mathematical article. It infers an edge from one result to another whenever the proof of the former refers to the latter through an ordinary cross-reference, requiring no manual annotation. The graph is written as a **Graphviz** `.dot` file, which can be rendered externally or, optionally, embedded into the document automatically.

1 Introduction

When writing a mathematical article, it is often useful to visualise how the results depend on one another: which lemma is used in the proof of which theorem, and so on. **proofgraph** builds such a dependency graph automatically. Each theorem-like environment becomes a node; each cross-reference (`\ref`, `\cref`, ...) appearing *inside a proof* becomes an edge from the proved result to the referred-to result.

2 Usage

Load the package after any theorem-providing package (such as `amsthm`) and *before* your `\newtheorem` declarations:

```
\usepackage{proofgraph}
\newtheorem{theorem}{Theorem}
\newtheorem{lemma}{Lemma}
```

Then write your document as usual. After two compilation runs, a file `\jobname.dot` is produced. Render it with **Graphviz**:

```
dot -Tpdf myfile.dot -o mygraph.pdf
```

`\uses` Inside a proof, `\uses{⟨label⟩}` records a dependency on the result labelled `⟨label⟩` even when no visible reference is made. A comma-separated list is accepted: `\uses{lem:a,lem:b}`.

A cross-reference in the *optional title* of a result is captured automatically as a dependency: `\begin{theorem}[(generalises~\ref{thm:x})]` in a result labelled `⟨A⟩` adds an edge from `⟨A⟩` to `thm:x`. This is handy for restatements (e.g. an introduction that previews results with `\begin{result}[(Corollary~\ref{cor:y})]`).

`\proofgraphedge` `\proofgraphedge{⟨A⟩}{⟨labels⟩}` adds edges recording that result `⟨A⟩` depends on the result(s) `⟨labels⟩` (a comma-separated list), like a `\uses` but stated outside any proof. This is handy for an “obvious” corollary that relies on earlier results without a `proof` environment: `\proofgraphedge{cor:x}{thm:y,lem:z}`. It may appear anywhere; all labels are resolved when the graph is written.

`\proofof` By default a proof is attached to the most recently started result. A detached proof whose optional title names its result is attached automatically to that result: writing `\begin{proof}` with the optional argument `[of Theorem~\ref{thm:x}]` attaches the proof to `thm:x`. This works for the `amsthm` `proof` environment and for `proof` defined as a theorem (the Springer classes); deferred proofs (as with `apxproof`) are likewise attached automatically. When none of this applies, `\proofof{⟨label⟩}` inside the proof pins it to the result labelled `⟨label⟩`.

`\proofgraphtrack` Result environments are detected automatically. Expository environments are excluded from the graph by default (`definition`, `example`, `remark`, `note`). In the preamble, `\proofgraphuntrack{⟨names⟩}` excludes further environment types and `\proofgraphtrack{⟨names⟩}` forces environments to be tracked (overriding the default exclusions), e.g. `\proofgraphuntrack{observation}` or `\proofgraphtrack{definition}`. Nodes are labelled by the result’s formatted name and number (“Theorem 1”), not by the environment name.

`\proofgraphignore` `\proofgraphignore{⟨from⟩}{⟨to⟩}` suppresses the edge from `⟨from⟩` to `⟨to⟩` (e.g. an unwanted forward citation).

`\proofgraphexclude` `\proofgraphexclude{⟨label⟩}` removes the result labelled `⟨label⟩`, together with all its incident edges, from the graph.

`\proofgraphstyle` `\proofgraphstyle{⟨env⟩}{⟨attrs⟩}` sets Graphviz node attributes for all results of the environment `⟨env⟩`, e.g. `\proofgraphstyle{lemma}{shape=ellipse,color=blue}`.

`\proofgraphstylecite` `\proofgraphstylecite{⟨attrs⟩}` sets the Graphviz attributes of the external citation nodes added by the `cite` option (default `shape=note`), e.g. `\proofgraphstylecite{shape=box,stroke=red}`.

`\proofgraph` In `autorun` mode, `\proofgraph` includes the rendered graph at that point (using the image produced on the previous run).

2.1 Options

option `selfloops=remove` (default) drops edges from a result to itself; `selfloops=keep` retains them.

option `autorun` (boolean, default false) runs Graphviz automatically at the end of the `autorun` run; requires compilation with shell-escape enabled.

option `engine` Graphviz layout engine (default `dot`).

option `engine` Output format for `autorun`/`\proofgraph` (default `pdf`).

option `direction=usedby` (default) orients an edge from the result that is used to the result that uses it, so an arrow `a->b` reads “a is used by b” (b relies on a).

option `direction=uses` reverses it (`a->b` means “a uses b”).

option `rankdir` Graph direction (default `LR`).

option `rankdir` `cite` (boolean, default false) also captures `\cite` inside proofs as dependencies on external work, drawn as distinct nodes. A `\cite` optional note is kept: `\cite[Thm~3]{key}` labels the node “Thm 3, ...”.

option `citelabel=key` (default) labels citation nodes by their `BIBTEX` key; `citelabel=tag` uses the printed citation tag instead (“[1]”, “[Knu74]”, “[Abiteboul et al. 1995]”, ...), recovered from the `.aux` (so it needs two runs). It supports the standard, `hyperref`-wrapped and `natbib` (numeric and author–year) bibliographies; if no clean tag can be recovered (e.g. before the first `BIBTEX` run) it keeps the key.

option Base name of the output file (default `\jobname`).
file

2.2 Use with `apxproof`

`apxproof` moves proofs to the appendix and typesets them there, so the references a proof makes are only executed at that later point. To attribute them to the right result, `proofgraph` relies on the hook `\apxproofhook`, which `apxproof` fires inside each deferred proof. This hook is available in `apxproof v1.4.1` and later; load the two packages in either order and no further setup is needed.

With an *earlier* `apxproof` the document still compiles cleanly and every result still becomes a node, but *no proof dependencies are captured* (the graph has nodes but no proof edges), because `proofgraph` cannot tell which appendix proof belongs to which result. In that case, add the dependencies manually with `\uses` (and `\proofof` to pin a proof to its result), or upgrade `apxproof`.

3 Limitations

Nodes are created only for environments declared through `\newtheorem` *after* the package is loaded. Results must be labelled for references to them to be resolved. Two compilation runs are required, as for any cross-reference. Theorem titles containing the characters " or \ in the node label are not escaped in this version.

4 Implementation

```
1 (*package)
2 \NeedsTeXFormat{LaTeX2e}[2005/12/01]
3 \ProvidesPackage{proofgraph}
4 [2026/05/31 v0.1.0 Dependency graph of results]
```

4.1 Dependencies

```
5 \RequirePackage{amsthm}
6 \RequirePackage{etoolbox}
7 \RequirePackage{xstring}
8 \RequirePackage{kvoptions}
9 \RequirePackage{graphicx}
```

`\pgph@trimsp` An expandable space-trimmer (labels inside a `\cite/\cref/\uses` list arrive with the space after each comma), built on the `expl3` primitive so it is robust to leading spaces and braces.

```
10 \ExplSyntaxOn
11 \cs_new:Npn \pgph@trimsp #1 { \tl_trim_spaces:n {#1} }
12 \ExplSyntaxOff
```

4.2 Options

```
13 \SetupKeyvalOptions{family=pgph,prefix=pgph@}
14 \DeclareStringOption[remove]{selfloops}
15 \DeclareBoolOption{autorun}
16 \DeclareStringOption[dot]{engine}
17 \DeclareStringOption[pdf]{format}
```

```

18 \DeclareStringOption[LR]{rankdir}
19 \DeclareStringOption[usedby]{direction}
20 \DeclareBoolOption{cite}
21 \DeclareStringOption[key]{citelabel}
22 \DeclareStringOption{file}
23 \ProcessKeyvalOptions*
24 \ifx\pgph@file\@empty\def\pgph@file{\jobname}\fi

```

4.3 Literal braces

We need catcode-12 braces to write the Graphviz digraph { } block.

```

25 \begingroup
26   \catcode'\[=1 \catcode'\]=2
27   \catcode'\{=12 \catcode'\}=12
28   \gdef\pgph@ob[{}]%
29   \gdef\pgph@cb[{}]%
30 \endgroup

```

4.4 Data structures

A global counter gives each result instance a unique numeric id. Nodes and edges are accumulated in list macros and emitted at end of document, so that editing commands (ignore, exclude, self-loops) can be applied to the complete graph regardless of where they appear.

```

31 \newcount\pgph@idcnt
32 \def\pgph@nodes{}
33 \def\pgph@edges{}
34 \def\pgph@ignores{}
35 \def\pgph@excludes{}
36 \let\pgph@curresult\@empty
37 \let\pgph@curproof\@empty

```

`\pgph@auxmap` The label-to-id map. `\label` inside a result records, both in memory and in the .aux file, that the user label maps to the current node id, so references resolve independently of hyperref.

```

38 \newcommand\pgph@auxmap[2]{\global\@namedef{pgph@map@#1}{#2}}
39 \newcommand\pgph@setmap[2]{%
40   \edef\pgph@tmpid{#2}%
41   \global\expandafter\edef\csname pgph@map@#1\endcsname{\pgph@tmpid}%
42   \protected@write\@auxout{}\string\pgph@auxmap{#1}{\pgph@tmpid}}%
43 }

```

4.5 Registering result environments

`\pgph@register` A result environment is tracked by installing a begin-hook that records a node. In every declaration command (`\newtheorem`, `\spnewtheorem`) the environment *name* is the first mandatory argument, so registration does not need to parse the remaining (varied) arguments. Each name met is queued as a *candidate* (deduplicated); the begin-hooks themselves are installed only at `\begin{document}`, once the inclusion and exclusion lists are settled, so the customisation commands below may appear anywhere in the preamble.

```

44 \let\pgph@candidates\@empty

```

```

45 \newcommand\pgph@register[1]{%
46   \edef\pgph@thisname{#1}%
47   \IfBeginWith\pgph@thisname{axp@}{-}{%
48     \ifcsname pgph@cand@\pgph@thisname\endcsname\else
49       \global\@namedef{pgph@cand@\pgph@thisname}{-}%
50       \ifx\pgph@candidates\@empty
51         \global\let\pgph@candidates\pgph@thisname
52       \else
53         \xdef\pgph@candidates{\pgph@candidates,\pgph@thisname}%
54       \fi
55     \fi
56   }%
57 }

```

We wrap `\newtheorem` (and, for Springer classes, `\spnewtheorem`) to register each newly declared environment, replaying the original command so its own argument parsing is untouched. The starred (unnumbered) forms are *not* auto-registered: a results graph wants numbered statements, and these forms are commonly used for unnumbered repeats of a result (e.g. the appendix copies produced by `apxproof` and `myproofs`), which would otherwise appear as spurious unnumbered duplicate nodes. Use `\proofgraphtrack` to track an unnumbered environment deliberately.

```

58 \let\pgph@orig@newtheorem\newtheorem
59 \renewcommand\newtheorem{\@ifstar{\pgph@nt@star}{\pgph@nt@plain}}
60 \newcommand\pgph@nt@plain[1]{\pgph@register{#1}\pgph@orig@newtheorem{#1}}
61 \newcommand\pgph@nt@star[1]{\pgph@orig@newtheorem*{#1}}
62 \ifdefined\spnewtheorem
63   \let\pgph@orig@spnewtheorem\spnewtheorem
64   \renewcommand\spnewtheorem{\@ifstar{\pgph@spnt@star}{\pgph@spnt@plain}}
65   \newcommand\pgph@spnt@plain[1]{\pgph@register{#1}\pgph@orig@spnewtheorem{#1}}
66   \newcommand\pgph@spnt@star[1]{\pgph@orig@spnewtheorem*{#1}}
67 \fi

```

```

\proofgraphtrack
\proofgraphuntrack
\proofgraphtrack

```

Result environments are detected automatically, but the analysis can be tuned. `\proofgraphtrack{⟨names⟩}` forces the listed environments to be tracked, overriding any exclusion; `\proofgraphuntrack{⟨names⟩}` excludes the listed environment *types*. Expository environments are excluded by default: `definition`, `example`, `remark`, `note`. Both commands belong in the preamble. `\pgph@defaultenvs` is the list of class-predefined names probed at `\begin{document}`; `\pgph@defaultexclude` seeds the exclusion set.

```

68 \newcommand\pgph@defaultenvs{theorem,lemma,proposition,corollary,definition,%
69   conjecture,claim,fact,remark,example,observation,notation,problem,%
70   question,property,assumption,hypothesis,principle}
71 \newcommand\pgph@defaultexclude{definition,example,remark,note}
72 \newcommand\proofgraphtrack[1]{%
73   \@for\pgph@one:=#1\do{%
74     \csundef{pgph@xtype@\pgph@one}%
75     \global\@namedef{pgph@force@\pgph@one}{-}%
76     \expandafter\pgph@register\expandafter{\pgph@one}%
77   }%
78 }
79 \newcommand\proofgraphuntrack[1]{%

```

```

80 \@for\pgph@one:=#1\do{%
81   \csundef{pgph@force@pgph@one}%
82   \global\@namedef{pgph@xtype@pgph@one}{}}%
83 }%
84 }
85 \expandafter\proofgraphuntrack\expandafter{\pgph@defaultexclude}

```

Environments predefined by the document class (acmart, lipics, Springer classes) are declared before the package is loaded, so the wrappers above never see them. At `\begin{document}` we add every common result name that exists to the candidate list, then install a begin-hook for each candidate that is forced or not excluded.

```

86 \newcommand\pgph@dohook[1]{%
87   \ifcsname pgph@hooked@#1\endcsname\else
88     \global\@namedef{pgph@hooked@#1}{}}%
89   \AtBeginEnvironment{#1}{\pgph@beginresult{#1}}%
90 \fi
91 }
92 \newcommand\pgph@maybehook[1]{%
93   \ifcsname pgph@force@#1\endcsname
94     \pgph@dohook{#1}%
95   \else
96     \ifcsname pgph@xtype@#1\endcsname\else\pgph@dohook{#1}\fi
97   \fi
98 }
99 \newcommand\pgph@resolveregistration[1]{%
100   \@for\pgph@one:=#1\do{%
101     \ifcsname\pgph@one\endcsname\expandafter\pgph@register\expandafter{\pgph@one}\fi}%
102   \ifx\pgph@candidates\empty\else
103     \@for\pgph@one:=\pgph@candidates\do{%
104       \expandafter\pgph@maybehook\expandafter{\pgph@one}}%
105     \fi
106 }
107 \AtBeginDocument{%
108   \expandafter\pgph@resolveregistration\expandafter{\pgph@defaultenvs}

```

`\pgph@beginresult` Start a result: allocate an id and record the node. The node is keyed and styled by `\pgph@hooknames` the environment name (third argument, also a fallback label), but displayed by the `\pgph@recordresult` result's *formatted* name (“Theorem”, “Lemma”), captured from the heading. We get that name by locally wrapping `\@begintheorem/\@opargbegintheorem` (the hooks every `amsthm-` and `\newtheorem`-based class routes its heading through), whose first argument is the title. These redefinitions are confined to the current environment group, so they revert at `\end`. We also locally redefine `\label` to populate the map.

Nodes are accumulated in *reverse* order of appearance (`\pgph@prependnode`). With `rankdir=LR`, Graphviz stacks disconnected components vertically in the order they are read, from the bottom up; emitting the results last-to-first therefore lays them out top-to-bottom in document order (first result on top), while connected results stay grouped. This is a layout tendency, not a guarantee, but needs no pack trickery.

```

109 \newcommand\pgph@prependnode[1]{%
110   \xdef\pgph@nodes{\unexpanded{#1}\unexpanded\expandafter{\pgph@nodes}}%
111 }

```

```

112 \newcommand\pgph@beginresult[1]{%
113   \global\advance\pgph@idcnt\@ne
114   \xdef\pgph@curresult{\the\pgph@idcnt}%
115   \protected@edef\pgph@tmp{%
116     \noexpand\pgph@node{\pgph@curresult}{#1}{#1}}%
117   \expandafter\pgph@prependnode\expandafter{\pgph@tmp}%
118   \pgph@hooknames
119   \ifx\label\pgph@maplabel\else\let\pgph@savedlabel\label\fi
120   \let\label\pgph@maplabel
121 }
122 \newcommand\pgph@hooknames{%
123   \ifdefined\@begintheorem
124     \let\pgph@orig@bt\@begintheorem
125     \def\@begintheorem##1##2{\pgph@recordresult{##1}{##2}\pgph@orig@bt{##1}{##2}}%
126   \fi
127   \ifdefined\@opargbegintheorem
128     \let\pgph@orig@obt\@opargbegintheorem
129     \def\@opargbegintheorem##1##2##3{%
130       \pgph@recordresult{##1}{##2}\pgph@scanrefs{##3}%
131       \pgph@orig@obt{##1}{##2}{##3}}%
132   \fi
133   \ifdefined\@spbegintheorem
134     \let\pgph@orig@spbt\@spbegintheorem
135     \def\@spbegintheorem##1##2##3##4{%
136       \pgph@recordresult{##1}{##2}\pgph@orig@spbt{##1}{##2}{##3}{##4}}%
137   \fi
138   \ifdefined\@spopargbegintheorem
139     \let\pgph@orig@spobt\@spopargbegintheorem
140     \def\@spopargbegintheorem##1##2##3##4##5{%
141       \pgph@recordresult{##1}{##2}\pgph@scanrefs{##3}%
142       \pgph@orig@spobt{##1}{##2}{##3}{##4}{##5}}%
143   \fi
144 }

```

\pgph@scanrefs A result whose optional title carries a cross-reference, e.g. `\begin{result}[(Corollary~\ref{cor:x})]` is understood to restate or build on that result: we record an edge from the current result to each label referenced in the title. We capture these by typesetting the title once into a discarded box with the reference-capturing commands installed and `\pgph@curproof` pointing at the current result; `\pgph@curproof` is saved and restored, so this affects nothing outside the title.

```

145 \newcommand\pgph@scanrefs[1]{%
146   \ifx\pgph@curresult\empty\else
147     \begingroup
148       \global\let\pgph@savedcurproof\pgph@curproof
149       \global\let\pgph@curproof\pgph@curresult
150       \pgph@install{ref}\pgph@install{cref}\pgph@install{Cref}%
151       \pgph@install{autoref}\pgph@install{eqref}\pgph@install{vref}%
152       \setbox\z@\hbox{#1}%
153       \global\let\pgph@curproof\pgph@savedcurproof
154     \endgroup
155   \fi
156 }

```

The formatted name is the first argument of every heading macro (`\@begintheorem`/`\@opargbegintheorem`).

for `amsthm/\newtheorem`, `\@spbegintheorem/\@spopargbegintheorem` for Springer's `\spnewtheorem`); the second argument is the number. We use that second argument only to tell *whether* the result is numbered (it is empty for the starred, unnumbered forms): an unnumbered result records no number, so it is dropped at emission unless tracked explicitly. The number *value*, when present, is read from `\@currentlabel` rather than from that argument, which some classes pollute (e.g. `lipics` injects `\advance\par@deathcycles\@ne`). `\@currentlabel`, set by the result's `\refstepcounter` just before the heading, is the clean number and needs no `\label`. Values are reduced to plain strings now, with fragile wrappers neutralized (notably `apxproof`'s forward-link). The first heading wins (guarded on the name), ignoring nested headings. The emptiness of the number argument is tested without expanding it, so a polluted (but non-empty) number does not trip the test.

```

157 \newcommand\pgph@recordresult[2]{%
158   \ifcsname pgph@name@\pgph@curresult\endcsname\else
159     \begingroup
160       \let\protect\@empty
161       \def\axp@forward@link##1##2{##2}%
162       \edef\pgph@tmpname{#1}%
163       \global\expandafter\let\csname pgph@name@\pgph@curresult\endcsname
164         \pgph@tmpname
165       \global\@namedef{pgph@seen@\pgph@curresult}{}%
166       \def\pgph@tmphn{#2}%
167       \ifx\pgph@tmphn\@empty\else
168         \edef\pgph@tmpnum{\@currentlabel}%
169         \global\expandafter\let\csname pgph@num@\pgph@curresult\endcsname
170           \pgph@tmpnum
171       \fi
172     \endgroup
173   \fi
174 }
```

`\pgph@maplabel` records the label-to-node mapping. As a fallback for classes whose headings pass through none of the hooked heading macros (so no heading was `seen`), it also captures the number from `\@currentlabel`. We do *not* do this once a heading was seen: there, an absent number means the result is genuinely unnumbered (a starred form), and `\@currentlabel` could be a stale value from an earlier result. The value is expanded to a plain string *now*, inside a group where fragile wrappers are neutralized, so that nothing dangerous survives to the emission pass: in particular, under `apxproof` forward-linking, `\@currentlabel` holds `\axp@forward@link{\(target)\}\{number\}`, whose hyper-link machinery must never be expanded in a non-typesetting context (it would run away). Reducing it to its number argument keeps it harmless.

`\pgph@maplabel`

```

175 \newcommand\pgph@maplabel[1]{%
176   \pgph@setmap{#1}{\pgph@curresult}%
177   \ifcsname pgph@num@\pgph@curresult\endcsname\else
178     \ifcsname pgph@seen@\pgph@curresult\endcsname\else
179       \begingroup
180         \let\protect\@empty
181         \def\axp@forward@link##1##2{##2}%

```



```

182     \edef\pgph@tmpnum{\@currentlabel}%
183     \global\expandafter\let\csname pgph@num@\pgph@curresult\endcsname\pgph@tmpnum
184     \endgroup
185     \fi
186 \fi
187 \pgph@savetlabel{#1}%
188 }

```

4.6 Hooking proofs

Inside a proof we redefine the reference commands to capture edges from the proved result.

```

189 \newcommand\pgph@adddge[2]{%
190   \protected@edef\pgph@tmpedge{\noexpand\pgph@edge{#1}{#2}}%
191   \expandafter\g@addto@macro\expandafter\pgph@edges\expandafter{\pgph@tmpedge}%
192 }
193 \newcommand\pgph@recordref[1]{%
194   \ifx\pgph@curproof\@empty\else
195     \@for\pgph@one:=#1\do{%
196       \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
197       \pgph@adddge{\pgph@curproof}{\pgph@k}%
198     }
199 }

```

`\pgph@beginproof` Attach the proof to the current result and install the capturing versions of the reference commands (only those that exist).

The capturing commands are `\protected` so that, when a reference appears in a moving argument inside a proof (e.g. a `\caption`), they do not expand into the `.aux/.lof` and leak internal tokens.

```

200 \protected\def\pgph@cap@ref#1{\pgph@recordref{#1}\pgph@savet@ref{#1}}
201 \protected\def\pgph@cap@cref#1{\pgph@recordref{#1}\pgph@savet@cref{#1}}
202 \protected\def\pgph@cap@Cref#1{\pgph@recordref{#1}\pgph@savet@Cref{#1}}
203 \protected\def\pgph@cap@autoref#1{\pgph@recordref{#1}\pgph@savet@autoref{#1}}
204 \protected\def\pgph@cap@eqref#1{\pgph@recordref{#1}\pgph@savet@eqref{#1}}
205 \protected\def\pgph@cap@vref#1{\pgph@recordref{#1}\pgph@savet@vref{#1}}
206 \newcommand\pgph@install[1]{%
207   \ifcsname #1\endcsname
208     \expandafter\ifx\csname #1\endcsname\csname pgph@cap@#1\endcsname
209     \else
210       \expandafter\let\csname pgph@savet@#1\endcsname\csname #1\endcsname
211       \expandafter\let\csname #1\endcsname\csname pgph@cap@#1\endcsname
212     \fi
213   \fi
214 }
215 \newcommand\pgph@beginproof{%
216   \let\pgph@curproof\pgph@curresult
217   \pgph@install{ref}\pgph@install{cref}\pgph@install{Cref}%
218   \pgph@install{autoref}\pgph@install{eqref}\pgph@install{vref}%
219   \ifpgph@cite\pgph@installcite\fi
220   \pgph@hookproofhead
221 }

```

When the `proof` environment is itself a theorem-like environment (as with Springer's `\spnewtheorem*{proof}`), its optional title is set by `\@opargbegintheorem/\@spopargbegintheorem`. We wrap these for the duration of the proof so that a title such as `[of Theorem~\ref{thm:x}]` sets `\proofof{thm:x}`, attributing the proof to its result.

```

222 \newcommand\pgph@hookproofhead{%
223   \ifdefined\@opargbegintheorem
224     \let\pgph@orig@obtp\@opargbegintheorem
225     \def\@opargbegintheorem##1##2##3{%
226       \pgph@scanproofarg{##3}\pgph@orig@obtp{##1}{##2}{##3}}%
227   \fi
228   \ifdefined\@spopargbegintheorem
229     \let\pgph@orig@spobtp\@spopargbegintheorem
230     \def\@spopargbegintheorem##1##2##3##4##5{%
231       \pgph@scanproofarg{##3}\pgph@orig@spobtp{##1}{##2}{##3}{##4}{##5}}%
232   \fi
233   \ifdefined\@Opargbegintheorem
234     \let\pgph@orig@Obtp\@Opargbegintheorem
235     \def\@Opargbegintheorem##1##2##3##4{%
236       \pgph@scanproofarg{##2}\pgph@orig@Obtp{##1}{##2}{##3}{##4}}%
237   \fi
238 }

```

`\pgph@scanproofarg` A proof whose optional title names its result (a detached proof) has its result given by that title, e.g. `thm:x` for an optional argument `[of Theorem~\ref{thm:x}]`, rather than by the most recently opened result. We capture this by wrapping the `proof` environment so that, when an optional title is given, the cross-reference it contains is turned into a `\proofof` (the title is typeset once into a discarded box with `\ref` and friends redefined). Without an optional title the original environment is used unchanged.

```

239 \newcommand\pgph@scanproofarg[1]{%
240   \begingroup
241     \let\protect\@empty
242     \def\ref##1{\proofof{##1}}\def\cref##1{\proofof{##1}}%
243     \def\Cref##1{\proofof{##1}}\def\autoref##1{\proofof{##1}}%
244     \setbox\z@\hbox{#1}%
245     \endgroup
246 }

```

To capture the optional title of a detached `amsthm` proof (`\begin{proof}[of Theorem~\ref{thm:x}]`) we redefine the outer `\proof` to read the optional argument and delegate to the original inner macro (saved as `\pgph@proofinner`), so no recursion occurs. We must do this *only* for a genuine optional-argument environment: a package that captures the proof body verbatim (`apxproof`, `myproofs`) redefines `proof` with no optional argument, and wrapping it would break that capture. We recognise the optional-argument form by the `\@protected@testopt` in its definition. When `proof` is instead a theorem-like environment (Springer `\spnewtheorem*{proof}`) its title is already handled by `\pgph@hookproofhead`, so here we just install the begin-hook.

```

247 \newcommand\pgph@wrapproof{%
248   \ifdefined\proof
249     \edef\pgph@proofmeaning{\meaning\proof}%
250     \edef\pgph@testoptstr{\detokenize{\@protected@testopt}}%
251     \IfSubStr\pgph@proofmeaning\pgph@testoptstr

```

```

252      {\pgph@redefproof}%
253      {\AtBeginEnvironment{proof}{\pgph@beginproof}}%
254    \fi
255  }
256  \newcommand\pgph@redefproof{%
257    \expandafter\let\expandafter\pgph@proofinner\csname\string\proof\endcsname
258    \def\proof{\pgph@beginproof
259      \@ifnextchar[{\pgph@proofopt}{\pgph@proofinner[\proofname]}}%
260    \def\pgph@proofopt[##1]{\pgph@scanproofarg{##1}\pgph@proofinner[##1]}%
261  }

```

Packages such as `apxproof` capture the body of a `proof` verbatim (to defer it to the appendix) and replay it there. Patching the `proof` environment then corrupts that capture, and the references are only executed at replay time anyway. So we wrap `proof` only when no such package is active; otherwise the deferred-proof capture (below) takes over. This happens at `\begin{document}`, once we know what is loaded.

```

262 \newif\ifpgph@apxproof
263 \AtBeginDocument{%
264   \@ifpackageloaded{apxproof}%
265     {\pgph@apxprooftrue\pgph@apxinit}%
266     {\pgph@wrapproof}%
267 }

```

`\pgph@apxinit` Under `apxproof`, deferred proofs are typeset in the appendix, where their references finally execute. `apxproof` fires `\apxproofhook` inside each such proof, passing the `\exp@r<n>` label of the proved theorem, which (because `apxproof` also attaches that label to the theorem in the main text) is already in our map. We resolve it to the source node and install the reference-capturing commands for the duration of the proof.

```

268 \newcommand\pgph@apxinit{\def\apxproofhook##1{\pgph@apxproofcapture{##1}}}
269 \newcommand\pgph@apxproofcapture[1]{%
270   \edef\pgph@curproof{\pgph@resolve{#1}}%
271   \ifx\pgph@curproof\@empty\else
272     \pgph@install{ref}\pgph@install{cref}\pgph@install{Cref}%
273     \pgph@install{autoref}\pgph@install{eqref}\pgph@install{vref}%
274     \ifpgph@cite\pgph@installcite\fi
275   \fi
276 }

```

`\pgph@hookcite` Optional `\cite` capture: external references become `cite:-`prefixed target labels, drawn later as distinct nodes. A `\cite[<note>]{<key>}`, is recorded for the key(s) and shown in the node label as “<note>, <key>”. The note is reduced to a plain string at capture time (fragile wrappers neutralized); the first note seen for a key wins.

```

277 \newcommand\pgph@cap@cite[2][ ]{%
278   \ifx\pgph@curproof\@empty\else
279     \def\pgph@citenotearg{##1}%
280     \@for\pgph@one:=#2\do{%
281       \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
282       \pgph@addedge{\pgph@curproof}{cite:\pgph@k}%
283       \ifx\pgph@citenotearg\@empty\else
284         \expandafter\pgph@recordcitenote\expandafter{\pgph@k}{##1}%

```

```

285     \fi}%
286 \fi
287 \pgph@samed@cite[#1]{#2}%
288 }
289 \newcommand\pgph@recordcitenote[2]{%
290   \ifcsname pgph@citenote@#1\endcsname\else
291     \begingroup
292       \pgph@citesanitize
293       \edef\pgph@tmpnote{#2}%
294       \global\expandafter\let\csname pgph@citenote@#1\endcsname\pgph@tmpnote
295     \endgroup
296 \fi
297 }
298 \newcommand\pgph@installcite{%
299   \ifcsname cite\endcsname
300     \let\pgph@samed@cite\cite
301     \let\cite\pgph@cap@cite
302 \fi
303 }

```

4.7 User commands

```

304 \newcommand\uses[1]{\pgph@recordref{#1}}
305 \newcommand\proofof[1]{%
306   \ifcsname pgph@map@#1\endcsname
307     \xdef\pgph@curproof{\csname pgph@map@#1\endcsname}%
308   \fi
309 }
310 \newcommand\proofgraphedge[2]{%
311   \g@addto@macro\pgph@edges{\pgph@labeledge{#1}{#2}}%
312 }
313 \newcommand\pgph@labeledge[2]{%
314   \edef\pgph@ef{\pgph@resolve{#1}}%
315   \ifx\pgph@ef\@empty\else
316     \@for\pgph@one:=#2\do{%
317       \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
318       \pgph@plainedge{\pgph@ef}{\pgph@k}}%
319   \fi
320 }
321 \newcommand\proofgraphignore[2]{%
322   \g@addto@macro\pgph@ignores{\pgph@ignorerule{#1}{#2}}%
323 }
324 \newcommand\proofgraphexclude[1]{%
325   \g@addto@macro\pgph@excludes{\pgph@excluderule{#1}}%
326 }
327 \newcommand\proofgraphstyle[2]{\global\@namedef{pgph@style@#1}{#2}}

```

External citation nodes (option cite) share one style, held in `\pgph@citestyle` and overridable with `\proofgraphstylecite`; it defaults to `shape=note`.

```

328 \newcommand\pgph@citestyle{shape=note}
329 \newcommand\proofgraphstylecite[1]{\renewcommand\pgph@citestyle{#1}}

```

4.8 Resolution helpers

These run at end of document, after the `.aux` has populated the map.

```

330 \newcommand\pgph@resolve[1]{%
331   \ifcsname pgph@map@#1\endcsname\csname pgph@map@#1\endcsname\fi
332 }
333 \newcommand\pgph@ignorerule[2]{%
334   \edef\pgph@f{\pgph@resolve{#1}}\edef\pgph@t{\pgph@resolve{#2}}%
335   \ifx\pgph@f\@empty\else\ifx\pgph@t\@empty\else
336     \@namedef{pgph@ig@\pgph@f @\pgph@t}{}%
337   \fi\fi
338 }
339 \newcommand\pgph@excluderule[1]{%
340   \edef\pgph@x{\pgph@resolve{#1}}%
341   \ifx\pgph@x\@empty\else\@namedef{pgph@ex@\pgph@x}{}\fi
342 }

```

4.9 Emission

`\pgph@node` Emit one node line, honouring exclusions and per-environment styling. External (`cite:`) targets are emitted separately as a pass over edges. An *unnumbered* result (no number captured) is omitted: a results graph is about numbered statements, and unnumbered theorem-like environments are typically expository or repeated copies (e.g. a class-predefined `\spnewtheorem*{claim}`). `\proofgraphtrack{<env>}` overrides this, so a deliberately unnumbered environment is still drawn.

```

343 \newcommand\pgph@node[3]{%
344   \ifcsname pgph@ex@#1\endcsname\else
345     \edef\pgph@nm{\ifcsname pgph@num@#1\endcsname\csname pgph@num@#1\endcsname\fi}%
346     \ifx\pgph@nm\@empty
347       \ifcsname pgph@force@#2\endcsname
348         \pgph@emitnode{#1}{#2}{#3}%
349       \else
350         \@namedef{pgph@ex@#1}{}%
351       \fi
352     \else
353       \pgph@emitnode{#1}{#2}{#3}%
354     \fi
355   \fi
356 }
357 \newcommand\pgph@emitnode[3]{%
358   \edef\pgph@s{\ifcsname pgph@style@#2\endcsname
359     , \csname pgph@style@#2\endcsname\fi}%
360   \edef\pgph@dn{\ifcsname pgph@name@#1\endcsname
361     \csname pgph@name@#1\endcsname\else#3\fi}%
362   \immediate\write\pgph@out{%
363     \space\space"n#1" [label="\pgph@dn\space\pgph@nm"\pgph@s];}%
364 }

```

`\pgph@edge` Emit one edge, resolving the target label to an id and applying the self-loop, ignore and exclude filters. Targets beginning with `cite:` denote external nodes.

```

365 \newcommand\pgph@edge[2]{%
366   \IfBeginWith{#2}{cite:}%
367     {\pgph@citeedge{#1}{#2}}%
368     {\pgph@plainedge{#1}{#2}}%
369 }

```

```

370 \newcommand\pgph@plainedge[2]{%
371   \edef\pgph@t{\pgph@resolve{#2}}%
372   \ifx\pgph@t\empty\else
373     \ifcsname pgph@ex@#1\endcsname\else
374     \ifcsname pgph@ex@\pgph@t\endcsname\else
375     \ifcsname pgph@ig@#1@\pgph@t\endcsname\else
376       \ifboolexpr{ test {\ifnumequal{#1}{\pgph@t}}
377         and test {\ifdefstring{\pgph@selfloops}{remove}} }%
378       {\pgph@emitedge{n#1}{n\pgph@t}}%
379     \fi\fi\fi
380   \fi
381 }

```

`\pgph@emitedge` The relation recorded is always “the result whose proof we are in (the first argument) uses the target (the second)”. The `direction` option chooses the arrow orientation: `direction=usedby` (default) draws *target to result*, so an arrow `a->b` reads “a is used by b” (i.e. b relies on a); `direction=uses` draws *result to target* (“a uses b”). The filters above work on the recorded relation, so they are unaffected by the chosen orientation. `\pgph@writeedge` deduplicates.

```

382 \newcommand\pgph@emitedge[2]{%
383   \ifdefstring{\pgph@direction}{uses}%
384   {\pgph@writeedge{#1}{#2}}%
385   {\pgph@writeedge{#2}{#1}}%
386 }
387 \newcommand\pgph@writeedge[2]{%
388   \ifcsname pgph@drawn@#1@#2\endcsname\else
389   \@namedef{pgph@drawn@#1@#2}{}%
390   \immediate\write\pgph@out{\space\space"#1" -> "#2";}%
391   \fi
392 }

```

`\pgph@citedge` A `cite:key` target: declare the external node once (keyed, as a stable id, by the `\pgph@setcitelabel` BIBTEX key), then draw the edge. The node *label* is built by `\pgph@setcitelabel`: it is the key by default, or, with `citelabel=tag`, the printed citation tag (the “[42]”/“[Knu74]” text) recovered from the `\b@{key}` macro that `\bibcite` writes to the `.aux` (so two runs are needed, and it falls back to the key when no tag is known, e.g. on the first run or with citation packages that do not populate `\b@{key}`). A `\cite` note, when present, is prepended as “(note), ...”.

```

393 \newcommand\pgph@citedge[2]{%
394   \ifcsname pgph@ex@#1\endcsname\else
395   \StrBehind{#2}{cite:}[\pgph@key]%
396   \ifcsname pgph@extnode@\pgph@key\endcsname\else
397   \@namedef{pgph@extnode@\pgph@key}{}%
398   \expandafter\pgph@setcitelabel\expandafter{\pgph@key}%
399   \immediate\write\pgph@out{%
400     \space\space"c@\pgph@key" [label="\pgph@clabel",\pgph@citestyle];}%
401   \fi
402   \pgph@emitedge{n#1}{c@\pgph@key}%
403   \fi
404 }
405 \edef\pgph@obraces{\expandafter\gobble\string\{ }
406 \newif\ifpgph@natnum
407 \newcommand\pgph@nil{ }

```

`\pgph@citesanitize` neutralizes the wrappers and spacing cruft that citation labels carry, so that an `\edef` of `\b@{key}` yields plain text: `hyperref's \hyper@@link` (keep its printed last argument) and the `\boxing/penalty/\ignorespaces` noise that `natbib` bakes into author lists.

```
408 \newcommand\pgph@citesanitize{%
409   \let\protect\empty
410   \def\hyper@@link[##1]##2##3##4{##4}%
411   \let\unskip\empty \let\ignorespaces\empty \let\unhbox\empty
412   \let\penalty\gobble \let\@M\empty \let\voidb@x\empty
413   \let\nobreakspace\space \let~\space
414   \def\hbox##1{##1}\def\mbox##1{##1}%
415 }
```

A `natbib \b@{key}` is `{\num}{\year}{\short}{\long}`. In numbers mode the tag is the number; in author–year mode it is the short author list and the year.

```
416 \def\pgph@natpick#1#2#3#4\pgph@nil{%
417   \ifpgph@natnum\def\pgph@x{#1}\else\def\pgph@x{#3 #2}\fi
418   \newcommand\pgph@natdo{\expandafter\pgph@natpick\pgph@x}{\}\pgph@nil}
419   \newcommand\pgph@setcitelabel[1]{%
420     \edef\pgph@cbase{#1}%
421     \ifdefstring{\pgph@citelabel}{tag}{%
422       \ifcsname b@#1\endcsname
423         \global\let\pgph@cbaseg\relax
424         \begingroup
425           \pgph@citesanitize
426           \edef\pgph@x{\csname b@#1\endcsname}%
427           \edef\pgph@dx{\detokenize\expandafter{\pgph@x}}%
428           \IfBeginWith\pgph@dx\pgph@obrace{%
429             \pgph@natnumfalse
430             \ifundefined{ifNAT@numbers}{\}%
431             \expandafter\ifx\csname ifNAT@numbers\endcsname\iftrue
432             \pgph@natnumtrue\fi}%
433           \pgph@natdo
434           \edef\pgph@dx{\detokenize\expandafter{\pgph@x}}%
435           \}\}%
436           \ifx\pgph@x\empty\else
437             \IfSubStr\pgph@dx\@backslashchar{\}%
438             \global\let\pgph@cbaseg\pgph@x}%
439         \fi
440       \endgroup
441       \ifx\pgph@cbaseg\relax\else\edef\pgph@cbase{[\pgph@cbaseg]}\fi
442     \fi
443   }\}%
444   \edef\pgph@clabel{%
445     \ifcsname pgph@citenote@#1\endcsname
446     \csname pgph@citenote@#1\endcsname, \fi
447     \pgph@cbase}%
448 }
```

`\pgph@emit` Open the file, resolve editing rules, write nodes then edges, close, and optionally run `Graphviz`.

```
449 \newwrite\pgph@out
450 \newcommand\pgph@emit{%
451   \immediate\openout\pgph@out=\pgph@file.dot\relax
```

```

452 \immediate\write\pgph@out{digraph proofgraph \pgph@ob}%
453 \immediate\write\pgph@out{\space\space rankdir="\pgph@rankdir";}%
454 \begingroup
455   \pgph@ignores
456   \pgph@excludes
457   \pgph@nodes
458   \pgph@edges
459 \endgroup
460 \immediate\write\pgph@out{\pgph@cb}%
461 \immediate\closeout\pgph@out
462 \ifpgph@autorun
463   \immediate\write18{\pgph@engine\space -T\pgph@format\space
464     \pgph@file.dot -o \pgph@file-proofgraph.\pgph@format}%
465 \fi
466 }

```

The emission is registered at `\begin{document}` rather than at load time so that it is added to the end-document hook *after* packages loaded later (notably `apxproof`, whose appendix, with the deferred proofs and their references, is built at end of document). This way the graph is written only once that material has been typeset, and the emission does not disturb the verbatim replay of deferred proofs.

```

467 \AtBeginDocument{\AtEndDocument{\pgph@emit}}

```

`\proofgraph` Include the rendered graph (from a previous run) in autorun mode.

```

468 \newcommand\proofgraph[1][]{%
469   \IfFileExists{\pgph@file-proofgraph.\pgph@format}%
470   {\includegraphics[#1]{\pgph@file-proofgraph.\pgph@format}}%
471   {\PackageWarning{proofgraph}{Rendered graph
472     \pgph@file-proofgraph.\pgph@format\space not found; compile with
473     shell-escape and the autorun option, then re-run}}%
474 }
475 \end{package}

```

Index

Numbers written in *italic* refer to the page where the corresponding entry is described; numbers underlined refer to the code line of the definition; numbers in roman refer to the code lines where the entry is used.

Symbols	49, 75, 82, 88,	\[	26
<code>\OM</code>	412	165, 327, 336,	\{ 27, 405
<code>\@Opargbegintheorem</code>		341, 350, 389, 397	\} 27
.... 233, 234, 235		<code>\@Opargbegintheorem</code>	\] 26
<code>\@auxout</code>	42	 127, 128,	
<code>\@backslashchar</code>	437	129, 223, 224, 225	A
<code>\@begintheorem</code>		<code>\@protected@testopt</code>	250
.... 123, 124, 125		<code>\@spbegintheorem</code>	268
<code>\@currentlabel</code>	168, 182	 133, 134, 135	<code>\AtBeginDocument</code> ..
<code>\@ifpackageloaded</code>	264	<code>\@spopargbegintheorem</code>	 107, 263, 467
<code>\@ifundefined</code>	430	 138, 139,	<code>\AtBeginEnvironment</code>
<code>\@namedef</code>	38,	140, 228, 229, 230	 89, 253
			<code>\AtEndDocument</code> 467

<code>\autoref</code>	243	<code>\newif</code>	262, 406	<code>\pgph@defaultexclude</code>	71, 85
<code>\xpf@forward@link</code> .		<code>\newtheorem</code>	58, 59	<code>\pgph@direction</code> . . .	383
	161, 181	<code>\nobreakspace</code>	413	<code>\pgph@dn</code>	360, 363
C		<code>\noexpand</code>	116, 190	<code>\pgph@dohook</code> . . .	86, 94, 96
<code>\catcode</code>	26, 27	P		<code>\pgph@dx</code>	
<code>\cite</code>	300, 301	<code>\PackageWarning</code> . . .	471		427, 428, 434, 437
<code>\Cref</code>	243	<code>\penalty</code>	412	<code>\pgph@edge</code> . . .	190, 365
<code>\cref</code>	242	<code>\pgph@addege</code>		<code>\pgph@edges</code>	
<code>\cs</code>	11		189, 197, 282		33, 191, 311, 458
<code>\csundef</code>	74, 81	<code>\pgph@apxinit</code>	265, 268	<code>\pgph@ef</code> . . .	314, 315, 318
D		<code>\pgph@apxproofcapture</code>	268	<code>\pgph@emit</code>	449
<code>\DeclareBoolOption</code> .			268	<code>\pgph@emitedge</code> . . .	
	15, 20	<code>\pgph@apxprooftrue</code> .	265		378, 382, 402
<code>\DeclareStringOption</code>		<code>\pgph@auxmap</code>	38	<code>\pgph@emitnode</code> . . .	
	14, 16,	<code>\pgph@beginproof</code> . .	200, 253, 258		348, 353, 357
	17, 18, 19, 21, 22	<code>\pgph@beginresult</code> .		<code>\pgph@engine</code>	463
<code>\detokenize</code>	250, 427, 434		89, 109	<code>\pgph@excluderule</code> .	
E		<code>\pgph@candidates</code> . . .	44,		325, 339
<code>\ExplSyntaxOff</code>	12		50, 51, 53, 102, 103	<code>\pgph@excludes</code> . . .	
<code>\ExplSyntaxOn</code>	10	<code>\pgph@cap@autoref</code> . .	203		35, 325, 456
H		<code>\pgph@cap@cite</code>	277, 301	<code>\pgph@f</code>	334, 335, 336
<code>\hbox</code>	152, 244, 414	<code>\pgph@cap@Cref</code>	202	<code>\pgph@file</code>	24, 451,
<code>\hyper@link</code>	410	<code>\pgph@cap@cref</code>	201		464, 469, 470, 472
I		<code>\pgph@cap@eqref</code> . . .	204	<code>\pgph@format</code>	463,
<code>\IfBeginWith</code>	47, 366, 428	<code>\pgph@cap@ref</code>	200		464, 469, 470, 472
<code>\ifboolexpr</code>	376	<code>\pgph@cap@vref</code>	205	<code>\pgph@hookcite</code>	277
<code>\ifdefstring</code>		<code>\pgph@cb</code>	29, 460	<code>\pgph@hooknames</code> . . .	109
	377, 383, 421	<code>\pgph@cbase</code>	420, 441, 447	<code>\pgph@hookproofhead</code>	
<code>\IfFileExists</code>	469	<code>\pgph@cbaseg</code>			220, 222
<code>\ifnumequal</code>	376		423, 438, 441	<code>\pgph@idcnt</code>	31, 113, 114
<code>\ifpgph@apxproof</code> . .	262	<code>\pgph@citeedge</code>	367, 393	<code>\pgph@ignorerule</code> . .	
<code>\ifpgph@autorun</code> . . .	462	<code>\pgph@citelabel</code> . . .	421		322, 333
<code>\ifpgph@cite</code>	219, 274	<code>\pgph@citenotearg</code> .		<code>\pgph@ignores</code>	
<code>\ifpgph@natnum</code>	406, 417		279, 283		34, 322, 455
<code>\IfSubStr</code>	251, 437	<code>\pgph@citesanitize</code> .		<code>\pgph@install</code>	
<code>\iftrue</code>	431		292, 408, 425		150, 151, 206,
<code>\ignorespaces</code>	411	<code>\pgph@citestyle</code> . . .			217, 218, 272, 273
<code>\includegraphics</code> . .	470		328, 329, 400	<code>\pgph@installcite</code> .	
J		<code>\pgph@clabel</code>	400, 444		219, 274, 298
<code>\jobname</code>	24	<code>\pgph@curproof</code>		<code>\pgph@k</code>	196, 197, 281,
L			37, 148,		282, 284, 317, 318
			149, 153, 194,	<code>\pgph@key</code>	395, 396,
			197, 216, 270,		397, 398, 400, 402
			271, 278, 282, 307	<code>\pgph@labeledge</code> . .	311, 313
<code>\label</code>	119, 120	<code>\pgph@curresult</code> . . .	36,	<code>\pgph@maplabel</code> . . .	
M			114, 116, 146,		119, 120, 175
<code>\mbox</code>	414		149, 158, 163,	<code>\pgph@maybehook</code> . .	92, 104
<code>\meaning</code>	249		165, 169, 176,	<code>\pgph@natdo</code>	418, 433
N			177, 178, 183, 216	<code>\pgph@natnumfalse</code> .	429
<code>\newcount</code>	31	<code>\pgph@defaultenvs</code> .		<code>\pgph@natnumtrue</code> . .	432
			68, 108	<code>\pgph@natpick</code> . . .	416, 418
				<code>\pgph@nil</code>	407, 416, 418
				<code>\pgph@nm</code>	345, 346, 363

<code>\pgph@node</code> . . .	116, 343	<code>\pgph@resolve</code>		<code>\pgph@x</code> . . .	340, 341,
<code>\pgph@nodes</code> 32, 110, 457			270, 314,		417, 418, 426,
<code>\pgph@nt@plain</code> . .	59, 60		330, 334, 340, 371		427, 434, 436, 438
<code>\pgph@nt@star</code> . . .	59, 61	<code>\pgph@resolverregistration</code>		<code>\ProcessKeyvalOptions</code>	
<code>\pgph@ob</code>	28, 452		99, 108		23
<code>\pgph@obrace</code> . .	405, 428	<code>\pgph@es</code>	358, 363	<code>\proof</code> 248, 249, 257, 258	
<code>\pgph@one</code> 73, 74, 75,		<code>\pgph@saved@autoref</code> 203		<code>\proofgraph</code>	2, 468
76, 80, 81, 82,		<code>\pgph@saved@cite</code> . .		<code>\proofgraphedge</code> . 2, 310	
100, 101, 103,			287, 300	<code>\proofgraphexclude</code> .	
104, 195, 196,		<code>\pgph@saved@Cref</code> . .	202		2, 324
280, 281, 316, 317		<code>\pgph@saved@cref</code> . .	201	<code>\proofgraphignore</code> 2, 321	
<code>\pgph@orig@bt</code> .	124, 125	<code>\pgph@saved@eqref</code> .	204	<code>\proofgraphstyle</code> 2, 327	
<code>\pgph@orig@newtheorem</code>		<code>\pgph@saved@ref</code> . . .	200	<code>\proofgraphstylecite</code>	
.	58, 60, 61	<code>\pgph@saved@vref</code> . .	205		2, 329
<code>\pgph@orig@obt</code> 128, 131		<code>\pgph@saved@curproof</code>		<code>\proofgraphtrack</code> 2, 5, 68	
<code>\pgph@orig@Obtp</code> 234, 236			148, 153	<code>\proofgraphuntrack</code> .	
<code>\pgph@orig@Obtp</code> 224, 226		<code>\pgph@saved@label</code> . .			2, 5, 68
<code>\pgph@orig@spbt</code> 134, 136			119, 187	<code>\proofname</code>	259
<code>\pgph@orig@spnewtheorem</code>		<code>\pgph@scanproofarg</code> .		<code>\proofof</code> 2, 242, 243, 305	
.	63, 65, 66	. 226, 231, 236, 239		<code>\protect</code>	
<code>\pgph@orig@spobt</code> . .		<code>\pgph@scanrefs</code>		. 160, 180, 241, 409	
.	139, 142		130, 141, 145	<code>\protected</code> 200, 201,	
<code>\pgph@orig@spobtp</code> .		<code>\pgph@selfloops</code> . . .	377	202, 203, 204, 205	
.	229, 231	<code>\pgph@setcitelabel</code> .	393	<code>\protected@edef</code> 115, 190	
<code>\pgph@out</code> . 362, 390,		<code>\pgph@setmap</code> . . .	39, 176	<code>\protected@write</code> . .	42
399, 449, 451,		<code>\pgph@spnt@plain</code> 64, 65			
452, 453, 460, 461		<code>\pgph@spnt@star</code> . 64, 66			
<code>\pgph@plainedge</code> . . .		<code>\pgph@t</code> . . . 334, 335,			
.	318, 368, 370	336, 371, 372,			
<code>\pgph@prependnode</code> .		374, 375, 376, 378			
.	109, 117	<code>\pgph@testoptstr</code> . .			
<code>\pgph@proofinner</code> . .			250, 251		
.	257, 259, 260	<code>\pgph@thisname</code> . 46,			
<code>\pgph@proofmeaning</code> .		47, 48, 49, 51, 53			
.	249, 251	<code>\pgph@tmp</code>	115, 117		
<code>\pgph@proofopt</code> 259, 260		<code>\pgph@tmpedge</code> . 190, 191			
<code>\pgph@rankdir</code>	453	<code>\pgph@tmphn</code> . . .	166, 167		
<code>\pgph@recordcitenote</code>		<code>\pgph@tmpid</code> . . 40, 41, 42			
.	284, 289	<code>\pgph@tmpname</code> . 162, 164			
<code>\pgph@recordref</code> 193,		<code>\pgph@tmpnote</code> . 293, 294			
200, 201, 202,		<code>\pgph@tmpnum</code>			
203, 204, 205, 304		. 168, 170, 182, 183			
<code>\pgph@recordresult</code> .	109	<code>\pgph@trimsp</code>			
<code>\pgph@redefproof</code> . .		. 10, 196, 281, 317			
.	252, 256	<code>\pgph@wrapproof</code> 239, 266			
<code>\pgph@register</code>		<code>\pgph@writeedge</code> . . .	382		
44, 60, 65, 76, 101					

				R	
				<code>\ref</code>	242
				S	
				<code>\setbox</code>	152, 244
				<code>\SetupKeyvalOptions</code>	13
				<code>\spnewtheorem</code> 62, 63, 64	
				<code>\StrBehind</code>	395
				T	
				<code>\tl</code>	11
				U	
				<code>\unexpanded</code>	110
				<code>\unhbox</code>	411
				<code>\unskip</code>	411
				<code>\uses</code>	1, 304
				V	
				<code>\voidb@x</code>	412
				Z	
				<code>\z@</code>	152, 244

Change History

v0.1.0
General: Initial version 1